



Universidad Nacional Autónoma de México

Facultad de Ciencias

Práctica Número 5

Cristian Guillermo Mejia Cisneros

Número de cuenta: 427329650

23 de octubre del 2026

1. Objetivos

- **Estructuras de repetición**: Usar los ciclos `for` y `while`, y distinguir en qué situación conviene cada uno.
- **Condicionales**: Usar `if, else if` y `else` para validar datos y elegir el proceso que corresponde a cada caso.
- **Bibliotecas**: Usar herramientas que `Java` ya trae incluidas, como `Math.random()`,`Math.pow()`, `Math.abs()` y `Math.PI` dentro de sus programas.
- **Casos de prueba**: Escoger valores que pongan a prueba las distintas ramas de un programa y verificar sus resultados contra valores esperados.
- **Organización de un proyecto**: Estructurar un proyecto separando el código fuente, los archivos compilados y la documentación. Además, compilar y ejecutar programas desde esa estructura.

2. Desarrollo

Parte 1: Calculando π a dardazos.

Actividad 1.1 - Un número fijo de dardos

Para esta parte de la practica, vamos a calcular el valor de π programando una formula basada en el método *Monte Carlo*, el cual es una técnica de aproximación, que utiliza varias situaciones o números al azar, analizando los resultados que estos arrojan para obtener la solución a un problema que en su totalidad, tiene un mayor grado de complejidad, pero visto como el conjunto de cada uno de los pequeños resultados, puede resolverse de forma más sencilla. Debe su nombre al famoso casino de Mónaco por su dependencia del azar y la probabilidad.

A pesar de que ya existe el valor "establecido" de un número tan famoso como lo es π, que fue calculado mediante otros muchos métodos, experimentar con *Monte Carlo* para obtener una estimación de π, nos ayuda a entender lo importante que llega a ser la programación, al momento de facilitar la resolución de problemas complejos.

Comenzando con la programación, utilizamos el siguiente hecho visto en Laboratorio:

$$\frac{dardos\ dentro\ del\ circulo}{dardos\ totales} \approx \frac{area\ del\ cuarto\ de\ circulo}{area\ del\ cuadrado} = \frac{\frac{\pi}{4}}{1} = \frac{\pi}{4} \tag{1}$$

Despejando π de **(1)**:

$$\pi \approx 4 * \frac{dardos\ dentro\ del\ circulo}{dardos\ totales} \tag{2}$$

Lo anterior no es más que la formula basada en el método *Monte Carlo*. Con esto en mente, para comenzar nuestro programa, declaramos las siguientes variables:

```
1    int dardosALanzar = 5000;
2    int dardosLanzados = 0;
3    int dardosDentro =0;
4    double abscisa;
5    double ordenada;
6    double distanciaAlOrigen;
```

```
7     double estimacionPi;
```

- `int dardosALanzar = 5000`: Guarda la cantidad de dardos que nosotros indicaremos cuantos podemos lanzar para hacer la aproximación, en este momento en especifico del código lo tenemos con 5000 pero puede cambiar según se necesite.
- `int dardosLanzados = 0`: Variable reservada para llevar el conteo de dardos lanzados, aunque talves de alguna manera puede realizarse el programa, unicamente con la variable `dardosALanzar` pensamos que era conveniente tener una variable exclusiva para llevar el conteo, en caso futuro (como veremos más adelante) de que no especifiquemos cuantos dardos vamos a lanzar. Se inicializa en cero para que después pueda ser usada recursivamente para llevar el conteo especificado.
- `int dardosDentro = 0`: Mismo caso que `dardosLanzados` solo que para llevar el conteo de los dardos que caen dentro del circulo.
- `double abscisa`: Si al cuadrado que sirve como el tablero donde caen los dardos, lo tomamos como un plano cartesiano, con $(0, 0)$ en el vértice inferior izquierdo, entonces esta variable nos ayudará a tomar el valor de la abscisa (eje "x") de la coordenada del punto donde cae el dardo.
- `double ordenada`: Mismo caso que la variable anterior solo que ahora para el eje "y".
- `double distanciaalorigen`: Reservada para ahí guardar el valor del calculo de la distancia al origen. Solo la declaramos pues posteriormente se le asignara un valor.
- `double estimacionPi`: Guarda la estimación de Pi que será el valor más relevante a imprimir en pantalla al final.

Realizado lo anterior, procedemos escribiendo la siguiente línea de código:

```
1     if (dardosALanzar > 0){
```

Es una línea de código que hemos colocado extra, por si en algún momento, necesitamos que a este programa, un usuario mediante la terminal ingrese la cantidad de dardos a lanzar, sea un valor válido. Sabemos el hecho de que no debemos agregar cosas extra si no se han pedido, pero pensamos conveniente agregar esta pequeña instrucción. Dicho esto, tenemos las siguientes lineas de código:

```
1     while(dardosLanzados < dardosALanzar){
2
3         abscisa = Math.random();
4         ordenada = Math.random();
```

En nuestro bloque de código anterior, tenemos un `while` que compara las variables `dardosALanzar` y `dardosLannzados`. Mientras los dardos lanzados sea estrictamente menor a la cantidad total de dardos que debemos lanzar, el ciclo seguirá repitiéndose. La comparación puede realizarse sin problema ya que ambas variables han sido inicializadas anteriormente, es debido a este mismo hecho que usamos un "<" y no un "<=", si colocáramos el menor o igual, realizaría 1001 lanzamientos.

Anteriormente, solo habíamos declarado `abscisa` y `ordenada`, debido al hecho de que en este último bloque de código es donde le asignamos su respectivo valor, utilizando la herramienta `Math.random`, sacada de la API (Apliccation Programming Interface) `Math` que no es más que

una "caja de herramientas" de Java, para realizar diversas operaciones; a lo largo de la práctica, será común utilizar distintas herramientas de `Math`.

`Math.random` genera valores entre 0 y 1 (sin tomar nunca el valor de 1), y nosotros mediante la asignación de las variables `abscisa` y `ordenada`, generamos coordenadas aleatorias donde posiblemente podría caer un dardo lanzado. Estos últimos datos nos ayudaran a calcular la distancia del punto de origen a esta coordenada generada, si esta distancia es menor o igual que 1 entonces el dardo esta dentro del circulo (pues el radio de esta circunferencia es 1), si es mayor a 1 entonces cae fuera del círculo. En la `Figura 1` vemos la misma gráfica vista en Laboratorio para entender un poco más la visualización de lo planteado anteriormente.

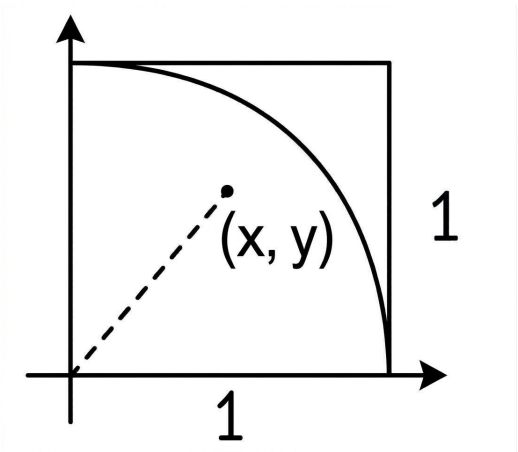


Figura 1: *Circulo representativo utilizado para calcular* π *mediante* **Monte Carlo**.

Como vamos a calcular la distancia al origen, tenemos que:

$$\sqrt{(x-0)^2+(y-0)^2} = \sqrt{x^2+y^2} \tag{3}$$

Luego, la distancia al origen debe ser menor o igual que uno, es decir:

$$\sqrt{x^2+y^2} \leq 1 \tag{4}$$

Elevando al cuadrado ambos lados de **(4)**:

$$x^2+y^2 \leq 1 \tag{5}$$

Por lo tanto solo debemos realizar x^2+y^2 y comparar si es menor o igual que 1, hecho por el cual solo escribimos:

```
1    distanciaAlOrigen = Math.pow(abscisa,2) + Math.pow(ordenada,2)
```

Donde nos ayudamos de `Math.pow` que es la herramienta que calcula un valor elevado a un exponente. La sintaxis de `Math.pow(z,n)` indica que al número z lo elevamos a n es decir z^n. Luego con:

```
1    if (distanciaAlOrigen <= 1.0){
2                dardosDentro += 1;
3                }
```

Verificamos si el dardo cae dentro del circulo, si se cumple, a la variable `dardosDentro` le sumamos uno, utilizamos `+=1` puesto que la variable ya habia sido inicializada en 0. En caso de que no este dentro del círculo, simplemente se salta la sentencia, suma uno a la variable

`dardosLanzados` para indicar que ya hemos lanzado un dardo y guarde cuantos hemos ya lanzado hasta el momento, para que en algún punto la sentencia de control `while` pueda detenerse.

Con los datos obtenidos una ves que se realice todo el ciclo `while`, calculamos π mediante:

```
    estimacionPi = 4.0 * dardosDentro/dardosLanzados;
```

En este caso no es necesario poner paréntesis, `4.0` nos ayudará a realizar un *casting* implícito, puesto que nuestras variables `dardosDentro` y `dardosLanzados` son del tipo **int**. Finalmente, imprimimos los resultados en pantalla:

```
    System.out.println("========ESTIMACION DE PI A DARDAZOS========\n")
       ;
System.out.println("Dardos lanzados: " + dardosLanzados);
System.out.println ("\nDardos que cayeron dentro del circulo: "+
   dardosDentro);
System.out.printf("\nEstimacion de Pi: %.6f", estimacionPi);
        }

else{

System.out.println ("No podemos lanzar una cantidad vacia o negatica de
    dardos");

        }

    }
}
```

El `else` final, es del `if` que utilizamos al principio para comprobar que no se colocará un valor no válido para una cantidad de dardos a lanzar. Utilizamos `printf` para poder dar la precisión de 6 digitos que es requerida. Todo el código en conjunto se ve de la siguiente manera:

```
    public class EstimacionPi {

public static void main (String [] args){

    int dardosALanzar = 5000;
    int dardosLanzados = 0;
    int dardosDentro =0;
    double abscisa;
    double ordenada;
    double distanciaAlOrigen;
    double estimacionPi;

if (dardosALanzar > 0){
    while(dardosLanzados < dardosALanzar){


        abscisa = Math.random();
        ordenada = Math.random();
        distanciaAlOrigen = Math.pow(abscisa,2) + Math.pow(ordenada,2);

            if (distanciaAlOrigen <= 1.0){
                dardosDentro += 1;
```

```
23
24                  }
25
26          dardosLanzados ++;
27
28              }
29
30  estimacionPi = 4.0 * dardosDentro/dardosLanzados;
31
32  System.out.println("========ESTIMACION DE PI A DARDAZOS========\n");
33  System.out.println("Dardos lanzados: " + dardosLanzados);
34  System.out.println ("\nDardos que cayeron dentro del circulo: "+
        dardosDentro);
35  System.out.printf("\nEstimacion de Pi: %.6f", estimacionPi);
36          }
37
38  else{
39
40  System.out.println ("No podemos lanzar una cantidad vacia o negatica de
        dardos");
41
42          }
43
44      }
45  }
```

Ahora para experimentar con el código hecho anteriormente, vamos a calcular 3 valores de π al lanzar $100, 1000, 10000, 100000$ y 1000000, los resultados a estos últimos calculos podemos verlos recolectados en la **Tabla 1**.

Num. de experimento	**Número de dardos**	**Estimación de π**
1	100	3.400000
2	100	3.160000
3	100	3.040000
4	1000	3.120000
5	1000	3.164000
6	1000	3.084000
7	10000	3.134000
8	10000	3.127200
9	10000	3.122000
10	100000	3.141920
11	100000	3.146040
12	100000	3.147720
13	1000000	3.142664
14	1000000	3.140200
15	1000000	3.141056

Tabla 1: Repeticiones del experimento con *Monte Carlo*, lanzando 100, 1000, 10000 100000 y 1000000

Con estos resultados podemos responder las siguientes preguntas:

1. ¿Por qué tres ejecuciones con el mismo número de dardos dan resultados distintos?

   Respuesta: Pensamos que es debido al hecho de que cada vez que ejecutamos nuevamente el programa, `Math.random` genera valores nuevos para las variables `abscisa` y `ordenada`, como son al azar, las coordenadas cambian y generan un resultado distinto de `distanciaAlOrigen` y en consecuencia un resultado distinto de dardos dentro del círculo.

2. ¿Qué pasa con la estimación conforme aumenta el número de dardos?

   Respuesta: En varios casos, la estimación mejora de manera significativa, aproximándose un poco más a π pues la cantidad de dígitos después de 3. es mayor. Para las primeras estimaciones, con pocos dardos, hay muchos dígitos en 0, conforme aumentamos los dardos lanzados, hay más dardos dentro del círculo (pues es probable que tengamos más debido a que se lanzaron muchos más dardos) lo que provoca finalmente, un mayor número de dígitos que se acercan a π.

Para terminar la **Actividad 1.1** en la **Figura** 2 hemos agregado una captura de la terminal, realizando los cálculos para el llenado de la **Tabla 1**.

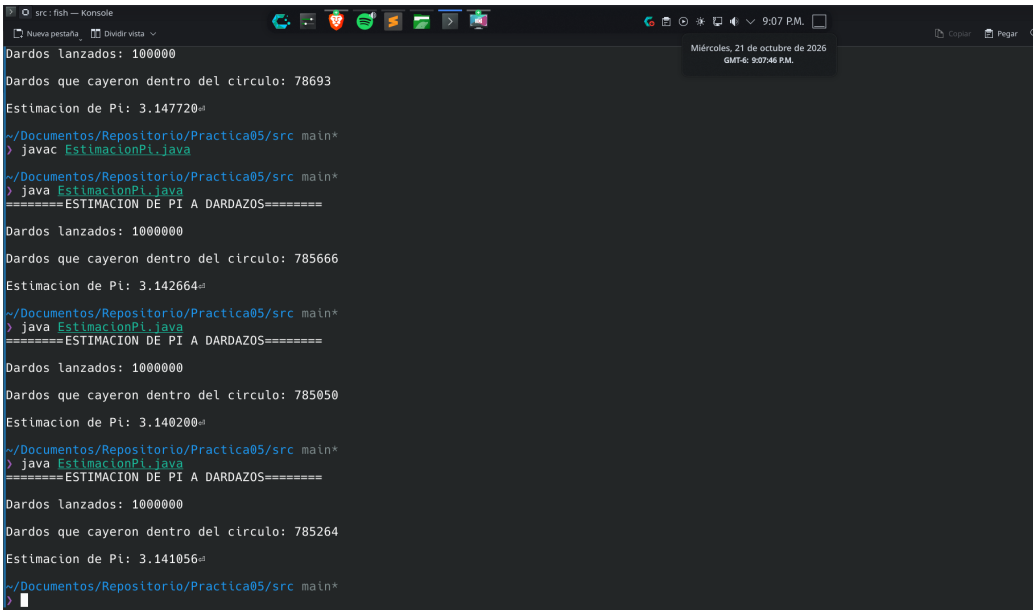


Figura 2: *Calculo de los resultados obtenidos en la* **Tabla 1**.

Actividad 1.2 - Repetir hasta llegar a la precisión deseada

En esta sección, escribir el código de un programa para calcular de nuevo π como lo hicimos en la **actividad 1.1**, solo que este ocasión, no vamos a determinar nosotros cuantos dardos vamos a lanzar, para este programa, se van a lanzar tantos dardos sean necesarios, para que la aproximación de π cumpla cierta tolerancia de error, es decir, que la estimación de π que obtengamos, tenga un error mínimo con respecto al valor establecido `Math.PI`, que es el valor reservado de las herramientas `Math`, con el valor más "exacto" de π. Para esto declaramos las siguientes variables:

```
1    int dardosLanzados = 0;
2    int dardosDentro =0;
3    double abscisa;
4    double ordenada;
```

```
    double distanciaAlOrigen;
    double toleranciaDePrecision = 0.1;
    double errorPi = toleranciaDePrecision;
    double estimacionPi=0.0;
```

Que son básicamente las mismas variables de la actividad pasada (quitando `dardosALanzar`), con otras variables agregadas y algunas modificaciones:

- `double toleranciaDePrecision = 0.1`: Una nueva variable que guarda, que tanto debe haber de error al comparar `estimacionPi` y `Math.PI`, más adelante en el código explicaremos a detalle a que nos referimos con lo anterior descrito. La variable la inicializamos según el valor que requerimos que sea la tolerancia, para este caso un 0.1.
- `double errorPi = toleranciaDePrecision`: Variable que guardara el valor de error de comparar `estimacionPi` con `Math.PI`, más adelante de igual manera se explicará como obtener este valor y porque lo inicializamos con la variable `toleranciaDePrecision`
- `double estimacionPi=0.0`: Es la misma variable que en el caso de la **Actividad 1.1**, solo que esta inicializada, esto debido al hecho, de que más adelante la metemos dentro del `while`, si no le asignamos un valor, al estar dentro de un condicional, **Java** pensará que no la inicializamos en ningún momento, aun cuando se cumpla la condición, por lo que marcará un error, motivo por el cual la inicializamos en 0.0, hecho que no afecta en nada al resultado pues será modificado hasta encontrar el valor esperado.

Para continuar, tenemos la siguiente parte del código, que es muy similar de nuevo a lo hecho en la anterior actividad:

```
    while(errorPi >= toleranciaDePrecision){
        abscisa = Math.random();
        ordenada = Math.random();

        distanciaAlOrigen = Math.pow(abscisa,2) + Math.pow(ordenada,2);



            if (distanciaAlOrigen <= 1.0){

                dardosDentro += 1;

                }
```

En el bloque anterior, el único cambio esta en el `while`, ahora la condicional no está basada en el hecho de que se termine de ejecutar, cuando lanza los dardos que le indicamos, ahora se ejecutará mientras **errorPi** sea mayor o igual a la tolerancia, donde este resultado está dado por:

$$errorPi = |estimacionPi - Math.PI| \tag{6}$$

Es por eso que `errorPi` lo inicializamos en `toleranciaDePrecision`, para que pueda hacer una comparación en `while`, puesto que si no la inicializamos no hay nada que comparar y no se compilará nuestro código. Al ser el error igual a la tolerancia, si o si, en la primera iteración, el `while` se ejecutará y la siguiente iteración dependerá de la actualización que se haga en `errorPi`, el hecho de asignarle un valor válido a `errorPi` para ejecutar `while`, hace que se vuelva una *variable centinela*.

Para continuar tenemos lo que resta de las instrucciones dentro del while:

```
    dardosLanzados ++;

        estimacionPi = 4.0 * dardosDentro/dardosLanzados;
        errorPi=Math.abs(estimacionPi-Math.PI);

            }
```

En lugar de hacer el cálculo `estimacionPi` fuera del `while` como en la actividad pasada, ahora está dentro de esta sentencia de control, debido al hecho de que `errorPi` necesita actualizarse constantemente, para en algún momento poder parar el ciclo y necesita forzosamente de nuevo valor de `estimacionPi` para conseguirlo. Lo anterior también explica el hecho de que antes de los cálculos de `estimacionPi` y `errorPi`, se encuentre:

```
    dardosLanzados ++;
```

Pues cada nuevo dardo tirado da otro resultado de **estimacionPi** y por consecuencia de `errorPi`, independientemente de si cae dentro del circulo un dardo. Para realizar el cálculo visto en la ecuación **(6)**, nos ayudamos de `Math.abs()` que saca el valor absoluto de lo que pongamos dentro del paréntesis, lo anteriror lo realizamos así pues queremos la distancia que hay entre los valores `estimacionPi` y `Math.PI`, dado que algún resultado puede dar negativo y no existen como tal las distancias negativas, requerimos que este valor sea positivo.

La impresión es la misma que en la actividad anterior, por lo que todo el código queda de la siguiente manera:

```
    public class PrecisionPi {

public static void main (String [] args){

    int dardosLanzados = 0;
    int dardosDentro =0;
    double abscisa;
    double ordenada;
    double distanciaAlOrigen;
    double toleranciaDePrecision = 0.1;
    double errorPi = toleranciaDePrecision;
    double estimacionPi=0.0;


    while(errorPi >= toleranciaDePrecision){
        abscisa = Math.random();
        ordenada = Math.random();

        distanciaAlOrigen = Math.pow(abscisa,2) + Math.pow(ordenada,2);



            if (distanciaAlOrigen <= 1.0){

                dardosDentro += 1;

                }
```

```
28          dardosLanzados ++;
29
30          estimacionPi = 4.0 * dardosDentro/dardosLanzados;
31          errorPi=Math.abs(estimacionPi-Math.PI);
32
33              }
34
35
36
37  System.out.println("========ESTIMACION DE PI A DARDAZOS========\n");
38  System.out.println("Dardos lanzados: " + dardosLanzados);
39  System.out.println ("\nDardos que cayeron dentro del circulo: "+
        dardosDentro);
40  System.out.printf("\nEstimacion de Pi: %.6f", estimacionPi);
41
42
43
44
45      }
46  }
```

Con el código escrito anteriormente, una vez compilado y ejecutado, vamos a realizar tres cálculos de π, para tres tolerancias, las cuales son 0,1, 0,01 y 0,001, podemos revisar el resultado de estos cálculos en la **Tabla 2**.

Tolerancia	**Ejecución**	**Dardos necesarios**	**Estimación de π**
0.1	1	9	3.111111
0.1	2	200	3.240000
0.1	3	5	3.200000
		Promedio:	3.183704
0.01	1	14	3.142857
0.01	2	14	3.142857
0.01	3	94	3.148936
		Promedio:	3.144883
0.001	1	270	3.140741
0.001	2	135	3.140741
0.001	3	359	3.142061
		Promedio:	3.141181

Tabla 2: Repeticiones del experimento con *Monte Carlo*, con tolerancia 0.1,0.01 y 0.001

En la **Figura** 3, vemos algunas de las aproximaciones calculadas ejecutando el programa realizado.

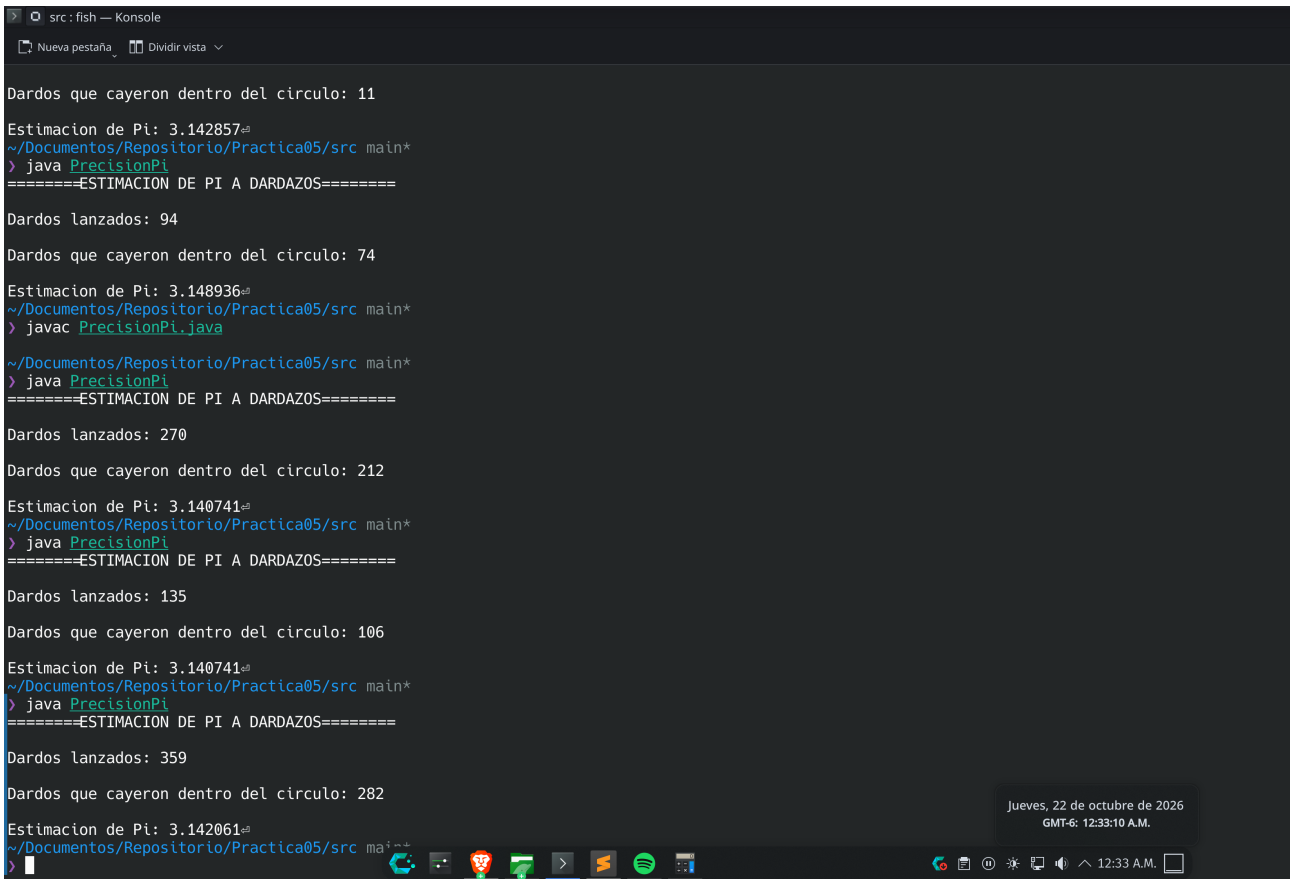


Figura 3: *Calculo de los resultados obtenidos en la* **Tabla 2**

En la tabla anterior, también agregamos una fila para calcular el promedio de los tres cálculos de π por cada tolerancia, lo cual nos ayudará a responder las siguientes preguntas:

1. ¿Qué tanto variaron los dardos necesarios entre las tres ejecuciones de una misma tolerancia?

   Respuesta: Para las tolerancias 0,1 y 0,01, dos resultados de dardos lanzados son relativamente parecidos, pero existía un valor que variaba bastante más que los otros. Para la tolerancia 0,01 los resultados son relativamente más congruentes, aunque varían más entre ellos que en las primeras dos tolerancias.

2. ¿Te parece que el promedio representa bien lo que paso?

   Respuesta: Para el caso de 0,1 parece a simple vista que el promedio se acerca más a una buena aproximación, debido a que da un resultado más acercado a π que los calculados, y más decimales después de 3., pero, en el fondo estamos pidiendo que la tolerancia sea 0,1, se nota bastante que el promedio a pesar de que lo cumple, no es congruente con lo obtenido para esta tolerancia, ya que dos de los resultados tienen muchos más dígitos cero, pues la restricción al error no era tan pequeña, e incluso varios dígitos repetidos.

   Para el caso de que la tolerancia sea mucho menor, el promedio se parece aun más a los tres resultados anteriormente calculados, podemos pensar que entre más restrictivos seamos con el error, el promedio da un valor aproximado de π más "exacto".

3. Con respecto a la **Tabla 1**, ¿Cuantos dardos necesitaste para llegar a un error menor a 0,001? ¿Concuerda con lo que observaste con un número fijo de dardos? si no concuerda ¿Por qué crees que pasa?

Respuesta: Observando los datos de la **Tabla 1**, para poder ver que valor de π estimado llega a un error menos de 0,001, nos ayudamos del jshell realizando operaciones del tipo `Math.abs(estimacionPi-Math.PI)` llegamos a que el valor que cumple con error menor a 0,001 es 3,141920, que necesito de una cantidad de 100000 dardos lanzados para llegar a la tolerancia correcta.

Este último resultado dista mucho de lo obtenido en la presente **Actividad 1.2** se piensa que es debido al hecho de que, en el código `PrecisionPi.java` el ciclo termina cuando ya conseguimos el error esperado a diferencia de `EstimacionPi.java`, que hace el resultado con todo los dados lanzados, no se detiene cuando obtiene un error menor, ejecuta todo independientemente si en algún momento había conseguido un valor con error esperado.

Parte 2: El regreso del señor Pines

En el archivo que se nos mando para resolver la práctica, es decir `P5_Decisiones_y_repeticiones.pdf`, a lo largo de un texto en la sección correspondiente a la **Parte 2**, el señor Pines que conocimos en la **Practica 4**, nos pide actualizar el cotizado que ya habíamos realizado anteriormente. En esta ocasión se solicita verificar que los datos ingresados sean correctos, que la clasificación de clientes ("P" = Preferente, "E" = Estándar y "R" = Riesgos) sirva para algo y que se calcule ahora el pago final y meses con un interés compuesto. Según lo indicado en esta práctica, se van a resolver estos problemas, seccionandolo por actividades.

Actividad 2.1

Para esta sección, antes de realizar lo del interés compuesto, vamos hacer los cálculos normales que ya habíamos realizado en la práctica pasada en `Cotizador.java`, solo que necesitamos que al asignarle a una variable un dato no válido (por ejemplo que los meses a calcular el interés sean negativos) el programa nos indique que no puede realizar los cálculos hasta obtener un dato válido y sumado a esto, nos indique, que dato es el que tiene una información errónea. Vamos a recordar las variables que teníamos anteriormente:

```
1     int precioCliente1 =12899;
2         String cliente1 = "Robbie Valentino";
3         char estadoCumplimientoCliente1 = 'E';
4         int mesesPagoCliente1 = 21;
5         double intereses=0.0;
```

- `int precioCLiente1 = 12899`: Precio de la computadora de Robbie y por tanto, del dinero que requiere pedir prestamo para pagarla, este dato será analizado para que no se ingrese, posteriormente, un valor no válido.
- `String cliente1 = "Robbie Valentino "`: Nombre del cliente, por el momento y con los conocimientos actuales, no vamos a analizar si este dato es valido, pues no afecta al calculo final de los intereses.
- `char estadoCumplimientoCLiente1 = ''`: Estado del cliente, si es un cliente que cumple, que no, o es regular, dato que será verificado para que sea válido.
- `int mesesPagoCliente1 = 21`: Los meses en que será pagado el préstamo, también será una variable a analizar si tiene un valor ingresado válido.
- `double intereses = 0.0`: interés simple a pagar, se inicializa en cero pues al estar dentro de un condicional, `Java` podría marcarlo como no inicializado.

Una ves mencionado las variables a utilizar, tenemos el siguiente código:

```
1     if (precioCliente1 > 0 && mesesPagoCliente1 >0 && (
          estadoCumplimientoCliente1 == 'E' ||  estadoCumplimientoCliente1
           == 'P' || estadoCumplimientoCliente1 == 'R')){
```

Hacemos una comparación de golpe, de que todos los datos se han ingresado correctamente, de ser el caso, ejecutamos lo que esta dentro del if:

```
1
2  System.out.println("========PRESTAMOS PINES========\n");
3
4
5   System.out.println("Estatus de "+ cliente1 + ": Cliente " +
       estadoCumplimientoCliente1);
6
7                  if (estadoCumplimientoCliente1 == 'P'){
8
9                      intereses = precioCliente1*(13.0/100)*(
                          mesesPagoCliente1/12.0);
10
11         }
12
13                 else if (estadoCumplimientoCliente1 == 'E'){
14
15                     intereses = precioCliente1*(15.0/100)*(
                          mesesPagoCliente1/12.0);
16
17         }
18
19                 else if (estadoCumplimientoCliente1 == 'R'){
20
21                     intereses = precioCliente1*(17.0/100)*(
                          mesesPagoCliente1/12.0);
22         }
```

tomando en cuenta las líneas de código, del formato anterior expuesto, de las lineas `1` a `6`, simplemente imprimimos un título y el estatus actual de cliente (cosa que podemos hacer pues el condicional tuvo un resultado exitoso), posteriormente con el condicional `else if`, calculamos los interés dependiendo del estatus del cliente, tanto si es **P**, **E** o **R**, se le baja 2 puntos a la tasa de interés, deja la tasa de interés sin modificación o la aumenta 2 puntos, respectivamente. En `Cotizador.java`, calculábamos el interés así:

```
1     intereses = precioCliente1*(15.0/100)*(mesesPagoCliente1/12.0);
```

Es decir, sin necesidad de reservar una variable para el interés 15 %, para este caso, solo sumamos o restamos 2, dependiendo del estatus, de nuevo, sin reservar una variable, es probable que si hay un cambio en los estatus o en el programa en general, seria preferente guardarlo en una variable en especifico, ya lo veremos más adelante en la siguiente actividad, pero por comodidad para esta parte, dejamos así el calculo. Analizado todo el bloque de código, tenemos lo siguiente aun dentro del if principal:

```
1  System.out.printf("\nDinero prestado: %d\n", precioCliente1);
2
3  System.out.printf("\nPago de intereses al final del plazo: %.2f\n",
      intereses);
```

```
4
5  System.out.printf("Total de monto a pagar: %.2f\n", precioCliente1+
       intereses);
6
7
8  System.out.printf("Pago por meses: %.2f\n", (precioCliente1+intereses)/
       mesesPagoCliente1);
9
10             }
```

Que solo son las impresiones de los datos que el cliente necesita saber, calculando algunos, como el total de monto a pagar y el pago por meses. Finalmente tenemos el `else`, del `if` que verificaba que todos los datos sean validos, en caso de que un dato ingresado sea erróneo:

```
1  else {
2
3      System.out.println("==DATOS NO VALIDOS==\n");
4      System.out.println("Detalles:\n");
5
6  if(precioCliente1 < 0){
7  System.out.println("-Valor no valido para la cantidad necesitada
       deprestamo.");
8          }
9
10 if(mesesPagoCliente1 < 0){
11 System.out.println("-Meses no validos para el pago del prestamo.\n");
12         }
13 if(estadoCumplimientoCliente1 != 'E' &&  estadoCumplimientoCliente1 !=
       'P' && estadoCumplimientoCliente1 != 'R'){
14
15 System.out.println("-No se ingreso un estado del cliente valido.\n");
16         }
17
18 }
```

El `else` funciona para especificar al usuario que algún detalle no es válido, en ves de ejecutar `else if` o `if - else`, colocamos varios if, en el caso de que si se ingresaron varios datos no válidos, especifique todos lo que están mal, al usar otra sentencia condicional, solo tomará la que se cumple, ignorara las otras y no imprimirá todos los datos que fueron ingresados de manera errónea.

Para comprobar que todo funciona correctamente, primero vamos a probar que si cambiamos el estatus del cliente, obtendremos distintos resultados. En la **Figura** 4, podemos observar los resultados ejecutados en la terminal para cada caso distinto.

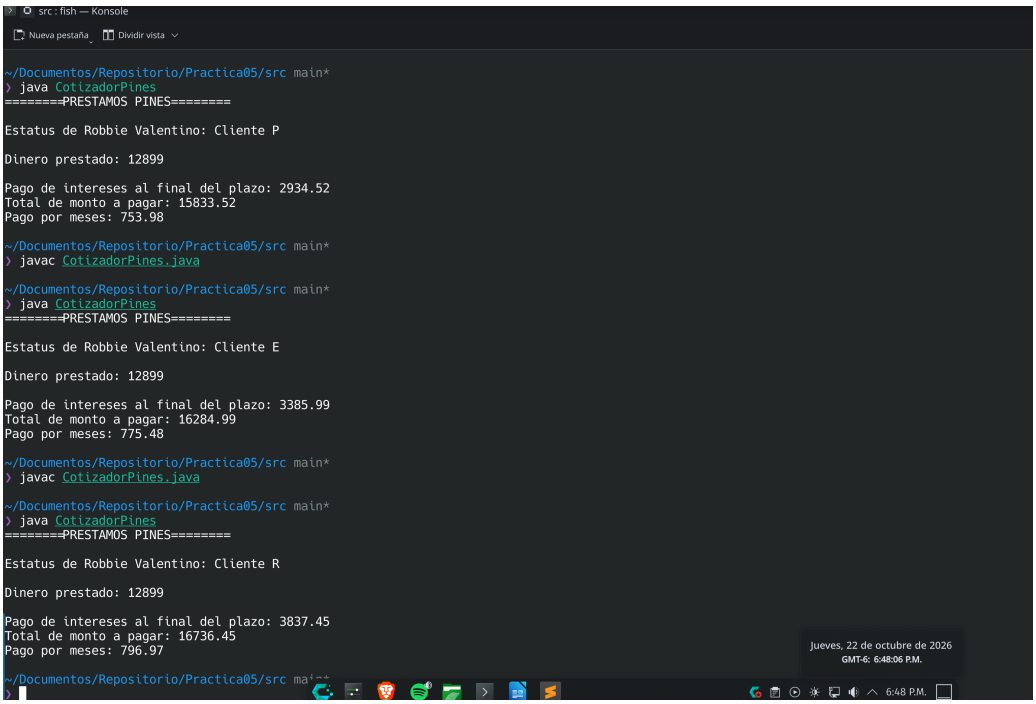


Figura 4: *Calculo de intereses para los distintos estatus posibles del cliente.*

Además de estas últimas pruebas, vamos a hacer `precioCliente1 = -12899` y `mesesPagoCliente1= -21`, para revisar que efectivamente, en la terminal se imprimirá, que los datos ingresados son erróneos. Después cambiamos nuevamente las variables anteriores a sus datos correctos, pero ahora hacemos `estadoCumplimientoCliente1 = 'A'`, que no es un estatus válido por lo que arrojará también errores, aunque sea la única variable que esta mal. Los resultados descritos con anterioridad podemos verlos en la **Figura** 5.

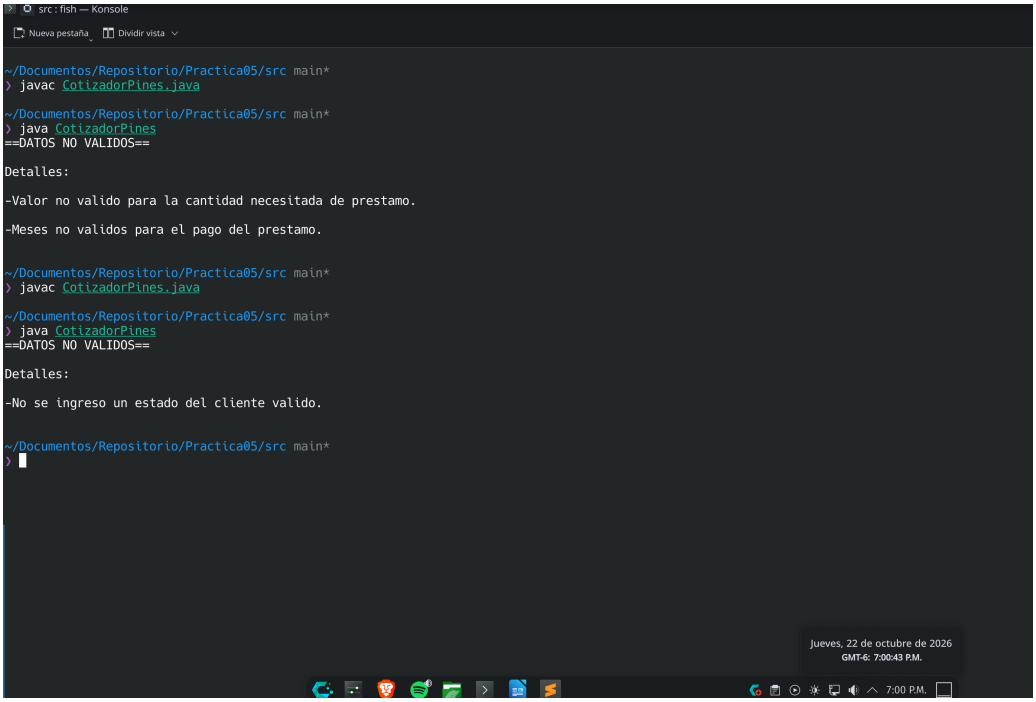


Figura 5: *Detección de datos no válidos para la cantidad de dinero prestada, el estatus de cliente y los meses a pagar*

Actividad 2.2

Basado en el archivo `CotizadorPines.java` de la actividad anterior, vamos a modificar el código para que en vez de solo calcular el interés simple, ahora también nos arroje al final el interés compuesto, que no es más que:

$$Cantidadprestada * (1 + tasamensual) \tag{7}$$

Donde la tasa mensual no es más que la tasa anual dividida entre 100 y luego entre 12, es decir, si nosotros tenemos 15 % de tasa de interés, entonces :

$$Cantidadprestada * \left(1 + \frac{15}{12 * 100}\right) \tag{8}$$

Para programar lo anterior, hicimos algunas modificaciones a nuestro código, comenzando con las nuevas variables:

```
1         double saldoPorMes=precioCliente1;
2         final double  TASA_DE_INTERES=15.0;
3         double tasaAnual=0.0;
```

- `double saldoPorMes=precioCliente1`: Este calcula el saldo de interés acumulado mes por mes, según los meses que acordamos que pagaríamos nuestra deuda. La inicializamos en `precioCliente1`, pues será la variable que se actualice en nuestro `for`, por lo que no modificamos nada acerca de la cantidad prestada.
- `final double TASA_DE_INTERES=15.0`: A diferencia de la actividad pasada, ahora definimos un valor constante para la tasa de interés, el cual es 15 % como lo establecimos al crear el programa, anteriormente no nos habíamos preocupado por declarar este valor, pero con las actualizaciones, como ya habíamos mencionado anteriormente, vemos conveniente tener un valor ya asignado.
- `double tasaAnual=0.0`: Esta variable, hace que no necesitemos modificar la variable tasa de interés (razón por la cual `TASA_DE_INTERES` es una constante) pues esta, se modificará dependiendo de los estatus que tenga cada cliente.

Lo que sigue en el código es prácticamente igual que en la **Actividad 2.1**, un `if - else` que verifique que los datos ingresados son validos, lo que cambia es lo que está dentro del `if`. En primer lugar cambiamos las condicionales que modificaban el interés dependiendo del tipo de cliente:

```
1     System.out.println("========PRESTAMOS PINES========\n");
2
3
4 System.out.println("Estatus de "+ cliente1 + ": Cliente " +
     estadoCumplimientoCliente1 + "\n");
5
6 System.out.println("
     ================================================================
     ");
7                 if (estadoCumplimientoCliente1 == 'P'){
8                         tasaAnual = (TASA_DE_INTERES-2.0)/100.0;
9
10        }
11
```

```
                else if (estadoCumplimientoCliente1 == 'E'){
                    tasaAnual = (TASA_DE_INTERES)/100.0;

        }

                else if (estadoCumplimientoCliente1 == 'R'){
                    tasaAnual = (TASA_DE_INTERES+2.0)/100.0;

        }
```

El bloque anterior de código, guarda en la variable `tasaAnual` la modificación que se requiera según el estatus del cliente, nos hace pensar que incluso podríamos haber hecho algo parecido en la actividad pasada, para ejecutar una única ves la impresión y no repetirlo en cada sentencia condicional.
Continuamos calculando el interés simple, para poder imprimirlo al final y verificar que con los datos obtenidos anteriormente de Robbie, los cálculos son correctos:

```
    intereses = precioCliente1*(tasaAnual)*(mesesPagoCliente1/12.0);
```

Seguido de la instrucción anterior, escribimos lo siguiente:

```
    for (int i=0; i<mesesPagoCliente1; i++){

                aldoPorMes *= (1 +  tasaAnual/12.0);
System.out.printf("En el mes %d se tiene un saldo de: %.2f\n",i+1,
   saldoPorMes);
            }
```

La sentencia de control `for` nos permitirá saber cuanto se va acumulando de interés compuesto mes con mes, actualizando `saldoPorMes` en cada iteración, así como `i`, hasta que haga imprima los 21 meses correspondientes a pagar según los datos de Robbie. Dentro de `printf` colocamos `i+1`, ya que `i` esta inicializado en 0, por lo que si solo colocáramos `i` y sin nada, en la primera iteración, imprimirá `... mes 0`, e irá un mes retrasado en cada impresión siguiente.

Ahora, veamos el bloque de código que sigue:

```
System.out.println("
   ==========================================================================\
   n");

 System.out.printf("\nTasa Aplicada %.2f porciento\n", TASA_DE_INTERES)
    ;

System.out.printf("\nPrestamo: $%d \n", precioCliente1);
System.out.printf("\nMeses de pago: %d\n", mesesPagoCliente1);


System.out.printf("\nInteres simple: $%.2f\n", intereses);


System.out.printf("\nTotal con interes simple: $%.2f\n", precioCliente1
    + intereses);

System.out.printf("\nInteres compuesto: $%.2f\n", saldoPorMes -
   precioCliente1);
```

Solo imprime los resultados de los datos requeridos a verificar que son correctos, pero antes de acabar con todo el condicional `if`, agregamos una última impresión, del saldo total a pagar con interés compuesto:

```
System.out.printf("\nTotal con interes compuesto: $%.2f\n",
   precioCliente1*Math.pow(1+ tasaAnual/12.0,21));
```

Este última instrucción, nos hace ver, que el saldo final con interés compuesto, no fue calculado de la misma manera, a como hicimos durante el `for`, la explicación es muy sencilla, dado que en cada iteración del `for`, multiplicábamos por $(1 + tasaAnual/12{,}0)$, al final, como hicimos 21 veces está multiplicación, es simplemente como si solo hubiéramos multiplicado por $(1 + tasaAnual/12{,}0)^{21}$, la cantidad de préstamo pedida, por lo que para calcular el saldo final de un interés compuesto, básicamente seguimos la formula:

$$saldofinal = Cantidadprestada * (1 + tasaAnual/12{,}0)^{plazo\ en\ meses} \tag{9}$$

Lo siguiente que sigue en el código es lo mismo que estaba dentro del `else` después del `if` inicial que comprueba que los datos tienen valores validos en la **Actividad 2.1**.

Según las instrucciones de la practica, debemos de obtener los resultado de la **Figura** 6. La **Figura** 7, muestra que en efecto, nuestro programa arroja los resultados esperados, así como una pequeña visualización de lo que implementamos en el `for`.

```
Tasa aplicada = 15%
Préstamo = $ 12899.00
Meses de pago = 21
Interés simple = $3385.99
Total con interés simple = $16284.99
Interés compuesto = $3844.71
Total con interés compuesto = $16743.71
```

Figura 6: *Datos que deberíamos obtener al terminar de programar el Cotizador*

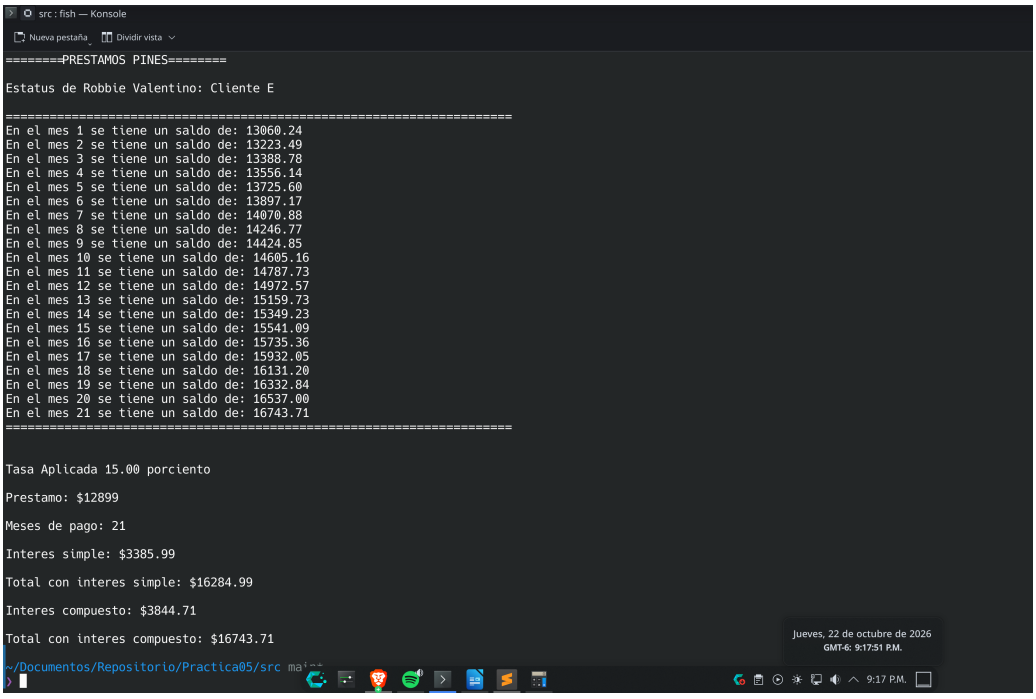


Figura 7: *Datos obtenidos al final de nuestro programa*

3. Conclusión

Lo realizado en esta práctica, nos ayudo a utilizar de manera correcta los ciclos `while` y `for`. Si bien `for` es un caso particular de `while`, hay momentos en que ocupar este ciclo, reduce de manera significativa el código escrito y facilita la manera de entender que estamos intentando realizar dentro de estas sentencias, sirve como un *Azúcar sintáctica*. El ciclo `for`, nos ayuda a repetir una acción varias veces, cuando tenemos cierta idea de cuando va a parar y `while` nos brinda apoyo, a ejecutar una serie de instrucciones especificas, haciendo más sencillo saber cuando parar, aun cuando no tenemos un limite.

Utilizamos las sentencias de condición, `if`, `else if` y `else`, para cumplir en su totalidad, las restricciones o condiciones requeridas en los programas, entiendo cual podemos utilizar en que momento y que valores podemos usar para evaluar condiciones.

Las herramientas de `Java`, del tipo textttMath, fueron determinantes para facilitar los cálculos en cada sección de la práctica, es claro que aunque nosotros mismos podríamos programar varios de los cálculos requeridos en las actividades, entender que tenemos estas herramientas a la mano, es decir, saber que alguien más ya hizo el trabajo "sucio", nos permite facilitar el escribir código que se usa con frecuencia, ahorrando lineas de código y el tiempo perdido realizando cada operación de manera manual.

En conjunto con lo anterior descrito, el realizar distintas actividades, que ponen a prueba nuestros conocimientos y capacidad de resolver problemas, nos ayuda a entender como organizarnos para analizar un problema, diseñar una solución, escribir el programa y probarlo, descartando los errores que se pudieron haber cometido durante el camino. Entender esto, es fundamental al momento de querer ser más organizados en cada proyecto nuevo que se nos propone y por consecuencia, llevar esta esta disciplina a cosas tan básicas como organizar de manera correcta nuestros archivos, para una correcta documentación.